

Guía de Laboratorio · Sesión 1

Tu primer contenedor — y por qué cambia todo

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida · Duración: 1 hora Plataforma: **Play with Docker** (navegador, gratis)

Hoy corres tu primer contenedor, publicas un servidor web real y compruebas con tus propias manos la propiedad más importante (y peligrosa) de los contenedores: son **efímeros**. Tu entregable: la **bitácora de la sesión**.

0. Requisitos (5 min — ideal traerlo listo)

1. Crea una cuenta **gratuita** en hub.docker.com (solo correo y contraseña).
2. Entra a labs.play-with-docker.com → **Login** (con tu cuenta de Docker Hub) → **Start**.
3. Clic en + **ADD NEW INSTANCE**. Aparece una terminal Linux con Docker listo.

⌚ La sesión de Play with Docker dura **máximo 4 horas** y al cerrar **se borra todo**. Por eso tu bitácora se escribe FUERA de la plataforma (documento aparte). Si la sesión muere a mitad de camino: crea otra instancia y repite los comandos — esa velocidad de reconstrucción es, precisamente, la gracia de los contenedores.

Parte 1 · Tu primer contenedor (20 min)

1.1 — El “hola mundo” de Docker:

```
docker run hello-world
```

Lee la salida: te cuenta exactamente qué acaba de pasar (el cliente habló con el motor, el motor bajó la imagen del registro, creó el contenedor y ejecutó el programa).

1.2 — Ahora algo real — un servidor web nginx:

```
docker run -d -p 8080:80 --name mi-web nginx
```

Pedazo	Qué hace
<code>-d</code>	lo corre en segundo plano (detached)
<code>-p 8080:80</code>	publica el puerto 80 del contenedor en el puerto 8080 del host
<code>--name mi-web</code>	le pone nombre (si no, Docker inventa uno)
<code>nginx</code>	la imagen oficial del servidor web

1.3 — Míralo vivo: en Play with Docker aparece un enlace **8080** arriba de la terminal — haz clic. Deberías ver **“Welcome to nginx!”**. Acabas de publicar un servidor web en menos de un minuto.

1.4 — Inspecciona lo que tienes:

```
docker ps          # contenedores corriendo
docker images      # imágenes descargadas
docker logs mi-web # los logs del servidor (verás tu propia visita)
```

👉 **Para la bitácora:** captura del navegador con “Welcome to nginx!” y captura de `docker ps`. ¿Cuánto tardó en levantar el servidor? ¿Qué apareció en los logs al visitarlo?

Parte 2 · Explora y comprueba lo efímero (25 min)

2.1 — Verifica ANTES de bajar (🔒 hábito de seguridad #1):

Abre hub.docker.com y busca `ubuntu`. Confirma el sello **«Docker Official Image»** antes de usarla. Esa verificación de 10 segundos es la diferencia entre una imagen mantenida y auditada, y la imagen de un desconocido.

2.2 — Entra a un contenedor interactivo:

```
docker run -it --name mi-ubuntu ubuntu bash
```

(`-it` = interactivo con terminal). Ahora estás **DENTRO** del contenedor. Compruébalo:

```
cat /etc/os-release    # eres Ubuntu (la máquina host es Alpine!)
ls /                   # su propio sistema de archivos
```

2.3 — Deja una huella:

```
echo "Colpensiones estuvo aquí - $(date)" > /huella.txt
cat /huella.txt
exit                               # sales del contenedor (esto lo detiene)
```

2.4 — El experimento clave. El contenedor quedó detenido (compruébalo con `docker ps -a`). Ahora elimínalo y vuelve a crear otro desde la MISMA imagen:

```
docker rm mi-ubuntu
docker run -it --name mi-ubuntu-2 ubuntu bash
cat /huella.txt          # ← ¿qué pasó?
exit
```

El archivo no existe. El contenedor nuevo nace de la imagen original, limpio. Todo lo que escribiste en el anterior murió con él.

🔥 **Para la bitácora:** ¿por qué desapareció `/huella.txt`? ¿Qué implicaría esto para una base de datos corriendo en un contenedor? (pista: lo resolvemos en la S4).

2.5 — Limpieza final:

```
docker stop mi-web && docker rm mi-web
docker rm mi-ubuntu-2
docker ps -a          # debe quedar vacío
```

Parte 3 · Bitácora — tu entregable (15 min)

Completa tu **bitácora de la Sesión 1** (documento aparte — plantilla al final) con:

1. **Evidencias:** las 2 capturas de la Parte 1 (nginx en el navegador + `docker ps`).
2. **Comandos:** los 5 comandos que consideres esenciales de hoy, con una línea de qué hace cada uno **en tus palabras**.
3. **Reflexiones** (3):
 - ¿Qué diferencia práctica viste entre *imagen* y *contenedor*?
 - ¿Por qué “efímero” es una ventaja Y un riesgo a la vez?
 - ¿Dónde le vería valor Colpensiones a levantar un servicio en segundos?

Checklist de la sesión

- ☐ Corrí `hello-world` y entendí el flujo cliente → motor → registro.
- ☐ Publiqué nginx con puerto y lo vi en el navegador.
- ☐ Verifiqué el sello de imagen oficial en Docker Hub antes del pull.
- ☐ Comprobé que el contenedor es efímero (la huella desapareció).
- ☐ Dejé el ambiente limpio (`docker ps -a` vacío).
- ☐ Entregué mi bitácora con capturas, comandos y reflexiones.

Plantilla de bitácora (copia esto en un documento)

BITÁCORA · SESIÓN 1 · 101-EAN GESTIÓN DE CONTENEDORES

Nombre: _____ Fecha: _____

1. EVIDENCIAS (pega tus 2 capturas)

2. MIS 5 COMANDOS ESENCIALES

comando → qué hace (en mis palabras)

1) ...

2) ...

3. REFLEXIONES

a) Imagen vs contenedor: ...

b) Efímero – ventaja y riesgo: ...

c) Valor para Colpensiones: ...

Próxima sesión: construyes **tu propia imagen** con un Dockerfile y la publicas en Docker Hub — tu software, empacado y listo para viajar entre local y nube. 🚀